МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ З ЛАБОРАТОРНОЇ
№5

на тему “Ліниві обчислення: генератори та itertools”
Варіант 15

Виконала:
Студентка 2 курсу
групи ФЕП-22
Сулятицька Христина

Перевірив:
Аспірант
Остріков Данило Андрійович

Львів-2026

Мета роботи

Ознайомитися з принципами лінивих обчислень у Python, генераторами та засобами модуля itertools. Реалізувати потік вимірювань через інжекцію callable-постачальника, ліниво відфільтрувати допустимі значення, обчислити рухомі середні та префіксні статистики. Реалізувати універсальні операції take, drop і sliding_window, створити детерміновані тести pytest та перевірити лінивість обчислень.

Теоретичні відомості

Ітератор — це об’єкт, який зберігає поточний стан обходу послідовності та повертає наступний елемент під час виклику next(). Після завершення послідовності генерується StopIteration. Ітерований об’єкт — це об’єкт, з якого можна отримати ітератор.

Лініві обчислення означають, що значення створюються лише тоді, коли споживач запитує наступний елемент. Такий підхід зменшує використання пам’яті та дає змогу працювати з великими й нескінченними потоками. У Python власні генератори реалізують за допомогою yield або yield from.

Модуль itertools містить інструменти для роботи з ітераторами. У роботі використано islice для обмеження кількості елементів, accumulate для накопичення префіксних сум, count для нумерації, а також takewhile і dropwhile для умовного відбору. Функція iter(callable, sentinel) послідовно викликає постачальник до моменту, коли він поверне sentinel.

Хід роботи

Для 15-го варіанта реалізовано потік випадкових вимірювань через інжекцію callable-постачальника. Постачальник передається функціям ззовні, тому джерело даних можна замінити без зміни логіки обробки. Для автоматизованих тестів використано детермінований provider, який повертає заздалегідь задану послідовність.

1. Створення структури проєкту

Проєкт розділено на модулі відповідно до їх призначення. У streams.py розміщено генератори та функції потокової обробки. У main.py реалізовано демонстраційний запуск. Автоматизовані перевірки розміщено в каталозі tests.

Файл	Призначення
streams.py	Потік вимірювань, фільтрація, take/drop, sliding_window, рухоме середнє та префіксна статистика.
main.py	Демонстрація роботи конвеєра та виведення результатів.
tests/test_streams.py	Перевірки коректності, крайніх випадків, sentinel і

	лінивості.
README.md, ASSIGNMENT.md	Опис варіанта, реалізації та інструкцій запуску.
requirements.txt, pyproject.toml, mypy.ini, .gitignore	Залежності та налаштування інструментів розробки.

Таблиця 1 — Структура проєкту

2. Реалізація потоку вимірювань у файлі streams.py

Функція measurement_stream(provider, sentinel) приймає callable-постачальник і використовує iter(provider, sentinel). Кожен виклик постачальника повертає наступне вимірювання. Коли повертається sentinel, ітерація завершується. Значення передається далі через yield у вигляді float, тому генератор не створює весь набір наперед.

Функція valid_measurements() виконує ліниву фільтрацію. За замовчуванням допустимими вважаються значення від −50 до 150 включно. Значення за межами діапазону відкидаються. Якщо нижня межа більша за верхню, функція повідомляє про помилку ValueError.

streams.py — генератори та конвеєр

```python
"""Лінива обробка потоку вимірювань через ін'єкцію callable-постачальника."""
from __future__ import annotations

from collections import deque
from itertools import accumulate, count, islice, takewhile, dropwhile
from typing import Callable, Iterable, Iterator, TypeVar

T = TypeVar("T")



def measurement_stream(provider: Callable[[], float], sentinel: object = None) -> Iterator[float]:
    """Повертає вимірювання від callable-постачальника до sentinel."""
    for value in iter(provider, sentinel):
        yield float(value)


def valid_measurements(
    source: Iterable[float], minimum: float = -50.0, maximum: float = 150.0
) -> Iterator[float]:
    """Ліниво пропускає значення поза допустимим діапазоном."""
    if minimum > maximum:
        raise ValueError("Мінімальна межа не може перевищувати максимальну.")
    for value in source:
        if minimum <= value <= maximum:
            yield value


def take(n: int, source: Iterable[T]) -> list[T]:
    """Повертає перші n елементів ітератора."""
    if n < 0:
        raise ValueError("n не може бути від'ємним.")
    return list(islice(source, n))


def drop(n: int, source: Iterable[T]) -> Iterator[T]:
    """Ліниво пропускає перші n елементів."""
    if n < 0:
        raise ValueError("n не може бути від'ємним.")
    iterator = iter(source)
    next(islice(iterator, n, n), None)
    return iterator
```

Рисунок 1 — Реалізація генераторного конвеєра

Функція bounded_measurements() поєднує отримання та фільтрацію даних і делегує обчислення через yield from. Таким чином, споживач отримує лише допустимі значення та не залежить від конкретного способу формування джерела.

3. Реалізація take, drop та sliding_window

Функція take(n, source) повертає перші n елементів за допомогою itertools.islice. Вона не запитує зайвих значень у джерела. Функція drop(n, source) пропускає перші n елементів і повертає ітератор, який продовжує обхід із наступного значення. Для від'ємного n передбачено ValueError, а n=0 не змінює джерело.

Функція sliding_window(source, k) формує послідовні вікна фіксованого розміру. Для зберігання поточного вікна використовується deque(maxlen=k), тому після додавання нового елемента найстаріший автоматично вилучається. Якщо джерело коротше за k, функція не повертає жодного вікна. Некоректний розмір $k \leq 0$ відхиляється.

streams.py — вікна та статистика

```
def sliding_window(source, k):
    if k <= 0:
        raise ValueError("k має бути додатним")
    iterator = iter(source)
    window = deque(maxlen=k)
    for _ in range(k):
        try:
            window.append(next(iterator))
        except StopIteration:
            return
    yield tuple(window)
    for value in iterator:
        window.append(value)
        yield tuple(window)

def moving_average(source, k):
    for window in sliding_window(source, k):
        yield sum(window) / k

def prefix_statistics(source):
    total = 0.0
    for number, value in zip(count(1), source):
        total += value
        yield number, total, total / number
```

Рисунок 2 — Ковзні вікна та потокові статистики

4. Рухоме середнє та префіксні статистики

Функція moving_average(source, k) проходить по вікнах, отриманих від sliding_window(), і повертає середнє арифметичне кожного вікна. Результат також є генератором, тому середні обчислюються лише під час запиту.

Функція prefix_sums() використовує itertools.accumulate для формування префіксних сум. Для потоку значень 21.0, 22.5, 23.0, 24.5, 25.0, 26.0, 27.5 результатом є 21.0, 43.5, 66.5, 91.0, 116.0, 142.0, 169.5. Функція prefix_statistics() повертає для кожного префікса кількість отриманих значень, їх суму та середнє. Для нумерації використовується itertools.count(1).

Додатково реалізовано below_threshold() і after_threshold() на основі takewhile та dropwhile. Ці функції демонструють умовне обмеження початкової частини потоку та пропуск початкових значень.

5. Демонстрація роботи програми

У main.py створено детермінований демонстраційний provider. Він повертає послідовність значень, серед яких є два недопустимі вимірювання: 999.0 та −100.0. Після фільтрації залишаються сім значень. Для них обчислюються рухомі середні з розміром вікна 3, префіксні суми та статистика кожного префікса.

```
Windows PowerShell — pytest та запуск main.py

PS C:\lab5_variant15> pytest -q
□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m.□[0m□[32m
□[32m□[32m□[1m11 passed□[0m□[32m in 0.04s□[0m□[0m

PS C:\lab5_variant15> python main.py
=== Лабораторна робота 5 — варіант 15 ===
Вихідний набір: [21.0, 22.5, 999.0, 23.0, 24.5, -100.0, 25.0, 26.0, 27.5]
1. Допустимі вимірювання: [21.0, 22.5, 23.0, 24.5, 25.0, 26.0, 27.5]
2. Рухоме середнє (вікно 3): [22.166666666666668, 23.333333333333332, 24.166666666666668, 25.166666666666668, 26.166666666666668]
3. Префіксні суми: [21.0, 43.5, 66.5, 91.0, 116.0, 142.0, 169.5]
4. Префіксна статистика (кількість, сума, середнє):
   (1, 21.0, 21.0)
   (2, 43.5, 21.75)
   (3, 66.5, 22.166666666666668)
   (4, 91.0, 22.75)
   (5, 116.0, 23.2)
   (6, 142.0, 23.666666666666668)
   (7, 169.5, 24.214285714285715)
5. Друге та наступні вікна: [(22.5, 23.0, 24.5), (23.0, 24.5, 25.0), (24.5, 25.0, 26.0)]
```

Рисунок 3 — Результат запуску тестів і демонстраційної програми

6. Створення та виконання тестів

Для перевірки реалізації створено автоматизовані тести pytest у файлі tests/test_streams.py. У тестах використано власний клас Provider, який повертає визначені значення та підраховує кількість викликів. Це дає змогу перевірити результати незалежно від випадковості та без використання зовнішніх джерел.

Тести перевіряють: отримання та фільтрацію вимірювань; завершення потоку за sentinel; лінивість постачальника; роботу take і drop на нескінченному count; обробку n=0 та від’ємних значень; ковзні вікна для звичайного, порожнього

та короткого джерела; некоректний розмір вікна; рухоме середнє; префіксні суми й статистику; роботу takewhile/dropwhile; перевірку меж фільтрації.

Під час контрольного запуску команда pytest -q завершилася успішно: усі 11 тестів пройшли. Тест лінивості підтвердив, що після запиту двох значень provider було викликано рівно двічі.

7. Перевірка крайніх випадків та лінивості

Окремо перевірено порожнє джерело, ситуацію, коли кількість елементів менша за розмір вікна, нульове значення n, від'ємні n та k, а також некоректні межі допустимого діапазону. Для нескінченного count усі приклади обмежуються take або islice, тому обчислення не зависають.

Лінивість забезпечується тим, що measurement_stream(), valid_measurements(), sliding_window(), moving_average(), prefix_statistics() та допоміжні обгортки повертають ітератори. Матеріалізація у список виконується лише в демонстрації або тестах, коли потрібен кінцевий результат.

8. Запуск проєкту та конфігурація

Для запуску в Windows необхідно створити віртуальне середовище, активувати його, встановити залежності та виконати тести й демонстраційну програму:

```
python -m venv .venv
.venv\\Scripts\\activate
pip install -r requirements.txt
pytest -q
python main.py
mypy .
black --check .
ruff check .
```

У requirements.txt зазначено pytest, mypy, black і ruff. У pyproject.toml визначено довжину рядка 88 символів та цільову версію Python 3.11. У mypy.ini ввімкнено строгий режим перевірки типів. У цьому звіті підтверджено запуск pytest; перевірки mypy, Black і Ruff слід виконати локально після встановлення залежностей.

Висновок: У ході лабораторної роботи я реалізувала 15-й варіант — потік вимірювань через інжекцію callable-постачальника. Я створила генератор отримання значень із завершенням за sentinel, реалізувала ліниву фільтрацію допустимого діапазону та побудувала функції take, drop і sliding_window. Для обробки даних я реалізувала рухоме середнє, префіксні суми через itertools.accumulate та префіксну статистику з використанням itertools.count. Під час тестування я перевірила коректність роботи конвеєра, обробку порожніх і коротких джерел, граничні значення параметрів та завершення потоку. Усі 11

тестів pytest завершилися успішно. Окремий тест підтвердив лінивість: постачальник виконав лише стільки викликів, скільки елементів запросив споживач. Таким чином, я закріпила навички роботи з генераторами, ітераторами та інструментами itertools і реалізувала потокову обробку даних без передчасної матеріалізації.